

ISIS Capitalist

Mini-projet dans le cadre du cours sur les architectures orientées services.

nicolas.singer@gmail.com



ISIS Capitalist

Présentation du projet

Principes du jeu « Adventure Capitalist »

Gameplay

Investir dans des produits et lancer leur production

Les managers

Les seuils (unlocks)

Les Cash Upgrades

Les anges

Les Angel Upgrades

Conception du serveur de mondes

Spécifications de l'API du serveur

Mise en place de la partie serveur

Mise en place du projet Angular

Description du travail attendu

Création des premiers composants

Réalisation du *layout* général

Éléments utiles de CSS

Les pipes d'Angular

Actions du joueur

Démarrage de la production d'un produit

La boucle principale de calcul du score

L'achat de produit

Les managers

Interface pour lister les managers

Engagement d'un manager

Afficher un message éphémère pour l'utilisateur

Badger les boutons pour informer le joueur

Prévenir le backend

Le nom du joueur

Les *unlocks*

Affichage des *unlocks*

Prise en compte des *unlocks* par le client

Les *Cash upgrades*

Affichage des *upgrades*

Prise en compte des *upgrades* par le client.

Gestion des anges

Prise en compte des anges dans les gains

Gestion des *Angel Upgrades*

Prise en compte des *angel upgrades* par le client.

Finalisation et branchement sur un autre monde

Présentation du projet

L'objectif de ce projet est de programmer un jeu vidéo calqué sur les mécanismes du jeu *free to play* « Adventure Capitalist » de la société Kongregate.

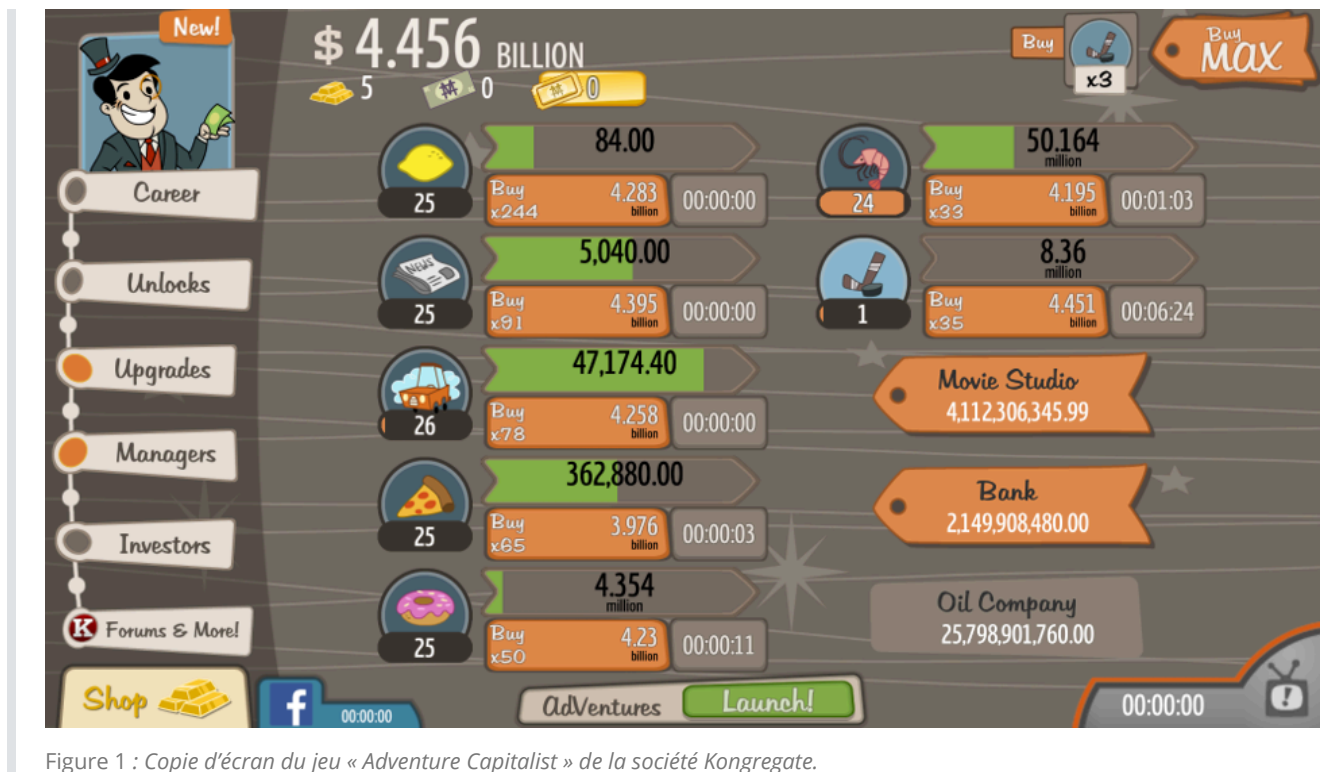


Figure 1 : Copie d'écran du jeu « Adventure Capitalist » de la société Kongregate.

Ce projet se composera d'une partie cliente (l'interface du jeu) et d'une partie serveur (la description du monde et sa persistance). Chaque groupe d'étudiants devra développer ces deux parties en s'appuyant sur un référentiel commun qui vise à assurer l'interopérabilité de chaque client avec chaque serveur. Tout comme le jeu d'origine permet au joueur de naviguer dans plusieurs mondes, le travail de chaque groupe consistera à définir un monde compatible avec le client des autres joueurs.

Principes du jeu « Adventure Capitalist »

« Adventure Capitalist » est un jeu satirique où l'objectif est d'augmenter à l'infini ses revenus en investissant dans la production de produits. Son originalité est de ne proposer aucun challenge, aucune énigme, aucun défi à relever. Vos rares actions et le temps qui passe vous feront automatiquement passer d'un vendeur de limonade à un méga-multi-hyper milliardaire, le tout sur fond de croissance hypnotique de dollars accumulés, les nombres obtenus ne pouvant plus s'exprimer à la fin qu'en multiple de puissances de dix.

Gameplay

Investir dans des produits et lancer leur production

Chaque niveau, que nous appellerons « monde », se compose d'un certain nombre de types de produits qui correspondent aux investissements réalisables. Ces investissements se traduisent par l'achat par le joueur d'un certain nombre d'exemplaires de ces produits.

Chaque type de produit est caractérisé par son temps de production, par le revenu procuré par la production d'un de ses exemplaires, et par le coût d'achat d'un exemplaire supplémentaire. Ce coût d'achat augmente en fonction du nombre d'exemplaires déjà possédés, car il se calcule sous la forme d'un pourcentage d'augmentation par rapport au prix du dernier exemplaire acheté. Ce pourcentage d'augmentation est propre à chaque type de produit.

Par exemple dans le monde « Terre » du jeu d'origine, la production d'une bouteille de limonade se fait en une demi-seconde et rapporte un dollar. Le premier exemplaire d'une telle bouteille coûte 4 dollars et ce coût se voit appliquer une augmentation de 7% à chaque unité supplémentaire. Acheter une deuxième bouteille coûte donc 4,28 ($1.07 * 4$) dollars, une troisième 4,58 ($1.07 * 4,28$) dollars, etc...

Le joueur lance la production d'un type de produit en cliquant sur l'icône qui lui correspond. Quand le temps de production est écoulé, l'argent du joueur se voit augmenter du nombre d'exemplaires du produit détenu multiplié par le revenu de ce type de produit. Ainsi si le joueur possède 4 bouteilles de limonade et qu'il lance leur production, au bout d'une demi-seconde, il aura gagné 4 ($4 * 1$) dollars.

Quand il dispose de la somme suffisante, le joueur peut acheter des exemplaires de produit supplémentaires, en cliquant sur le bouton « Buy » attaché au produit.



Figure 2 : Ici, le joueur possède 25 bouteilles de limonade qui lui rapportent 84 dollars à chaque production. Acheter une bouteille de plus coûte 20,29 dollars.

Pour faciliter la vie du joueur, le jeu propose un bouton qui permet d'acheter plusieurs exemplaires d'un produit d'un coup, par dix, par cent, ou selon la quantité maximale achetable en fonction de l'argent actuel du joueur.



Figure 3 : Quand le bouton de quantité d'achat est sur max, l'interface affiche combien le joueur peut acheter d'exemplaire d'un produit (ici 246, pour un coût total de 4,904 milliards).

Au fur et à mesure de ses gains, le joueur peut débloquer des produits supplémentaires et ainsi lancer la production de plusieurs types de produit en même temps.

Les managers

Chaque produit dispose d'un manager qui permet d'automatiser sa production. Au début de la partie, ce manager est inactif et peut-être débloqué contre une certaine somme d'argent, ce qui dispense ensuite le joueur de cliquer pour lancer la production de ce produit. Ce qui est intéressant c'est que l'automatisation de la production persiste même quand le joueur n'est plus connecté. Ainsi le compte en banque du joueur augmente automatiquement en fonction du temps qui passe. L'achat d'un manager se fait en cliquant sur le bouton « manager » de l'interface.



Figure 4 : Liste des managers avec leur nom, leur coût d'achat et le produit dont ils s'occupent.

Les seuils (unlocks)

Le revenu généré par un type de produit peut être boosté lorsque le joueur en obtient une quantité dépassant un certain seuil. Ce *boost* peut prendre la forme d'une augmentation de la vitesse de production ou d'une augmentation du revenu généré par le produit, sous la forme d'un ratio multiplicateur. La liste des seuils attachés à chaque produit est visualisable en cliquant sur le bouton « unlocks » de l'interface (Figure 5).

S'ajoutent à ces seuils par produit, des seuils globaux qui génèrent des bonus supplémentaires quand ils sont atteints par l'ensemble des produits. Par exemple sur la figure 5, on voit qu'atteindre une quantité de 25 pour chaque produit, double la production de chaque produit.



Figure 5 : liste des prochains bonus associés à l'obtention d'une certaine quantité d'un type de produit.

Les Cash Upgrades

Parallèlement aux bonus de seuil, le joueur a également la possibilité d'acheter des « upgrades » qui permettent eux aussi d'augmenter les revenus. Leur liste s'obtient par le bouton « upgrades ». Certains de ces bonus augmentent la production d'un seul produit alors que d'autres peuvent agir sur tous. Notons que certains *upgrades* peuvent également augmenter l'efficacité des anges (voir la prochaine section pour la signification de ces anges).

Les anges

Au fil du jeu le joueur va accumuler un certain nombre d'« Angel Investors ». Ces anges, quand ils sont actifs, offrent chacun un bonus de production de 2%. Par conséquent 50 anges actifs procurent un doublement des revenus ($2 \times 50 = 100\%$). Le problème, et finalement le sel du *gameplay*, c'est que les anges obtenus ne deviennent actifs qu'après une remise à zéro (*reset*) de la partie. Ce mécanisme oblige le joueur à repartir de zéro s'il veut bénéficier du bonus apporté par les anges.

Plus précisément, dans le jeu d'origine et sur le monde « terre », le nombre d'anges obtenus est lié aux gains cumulés du joueur depuis qu'il a commencé à jouer, selon la formule :

$$\text{nombre d'anges} = 150 \times \sqrt{\frac{\text{gains cumulés}}{10^{15}}}$$

A chaque reset de la partie, c'est ce nombre d'anges qui devient actif (moins ceux qui ont été dépensés dans les « Angel Upgrades », voir plus bas).

Cette formule traduit le fait qu'il faut de plus en plus d'argent pour accumuler des anges supplémentaires, mais aussi le fait que les anges persistent entre les *reset* puisque leur nombre dépend des gains accumulés par le joueur lors de chacune de ses parties.

Les Angel Upgrades

Les « Angel Upgrades » ont le même effet que les « Cash Upgrades » à ceci près que leur monnaie d'achat est l'ange au lieu d'être le dollar (Figure 6). Ainsi le joueur peut dépenser un certain nombre de ses anges actifs pour acheter des bonus de production (et parfois une certaine quantité de produits). Attention à bien réfléchir à ce type d'achat, car les anges dépensés sont perdus, et on ne profite plus de leur bonus de 2% de production supplémentaire.



Figure 6 – Bonus possibles en dépensant des anges pour le monde « terre ».

A titre d'exemple, voici les valeurs de certains paramètres du monde « terre » du jeu d'origine :

Produit	Revenu initial	Coût du premier exemplaire	Croissance du coût par exemplaire	Temps initial de production	Coût du manager
Limonade	\$1	\$4	7%	0,5 s	\$1 000
Journal	\$60	\$60	15%	3 s	\$15 000
Lavomatic	\$540	\$720	14%	6 s	\$100 000
Pizza	\$4 320	\$8 640	13%	12 s	\$500 000
Donut	\$51 840	\$103 680	12%	24 s	\$1 200 000
Crevette	\$622 080	\$1 244 160	11%	1 min 36 s	\$10 000 000
Hockey	\$7 464 000	\$14 929 920	10%	6 min 24 s	\$111 111 111
Film	\$89 579 000	\$179 159 040	9%	25 min 36 s	\$555 555 555

Conception du serveur de mondes

Spécifications de l'API du serveur

Comme nous l'avons dit, le logiciel complet sera composé d'une partie serveur et d'une partie cliente. La partie serveur a pour responsabilité de définir le monde proposé à la partie cliente, sous la forme des produits dans lesquels le joueur pourra investir, de leurs caractéristiques (cout, revenu, etc.), de leurs managers, de leurs upgrades et de leurs seuils, etc.

Le serveur aura également pour rôle de conserver la persistance des parties de chaque joueur s'y connectant.

Pour que les serveurs soient compatibles les uns avec les autres (en fait compatibles avec les clients qui s'y connecteront), nous devons définir un référentiel d'interopérabilité qui spécifie les protocoles de communication entre le serveur et les clients, ainsi que le format des messages échangés.

En termes de **protocoles**, nous utiliserons une communication client-serveur basée sur un service web au format GraphQL. Les points d'accès de ce service seront :

`getworld` : retourne au client une représentation complète de l'état actuel du monde sous la forme d'une entité de type « monde ».

`acheterQtProduit` : permet au client de communiquer au serveur qu'une certaine quantité d'un produit est achetée.

`lancerProductionProduit` : permet au client de communiquer au serveur qu'un produit vient d'être mis en production.

`engagerManager` : permet au client de communiquer au serveur l'achat du manager d'un produit.

`acheterCachUpgrade` : permet au client de communiquer au serveur l'achat d'un *Cash Upgrade*.

`acheterAngelUpgrade` : permet au client de communiquer au serveur l'achat d'un *Angel Upgrade*.

`resetworld` : permet au client de demander le *reset* du monde.

Voici le schéma GraphQL que vous devrez utiliser (ce schéma est repris dans l'énoncé du backend et sera utilisé pour générer vos classes métiers) :

```
enum RatioType {
  gain
  vitesse
  ange
}

type Palier {
  name: String!
  logo: String
  seuil: Float
  idcible: Int
  ratio: Int
  typeratio: RatioType
  unlocked: Boolean
}

type Product {
  id: Int!
  name: String
  logo: String
  cout: Float
  croissance: Float
  revenu: Float
  vitesse: Int
  quantite: Int
  timeleft: Int
  managerUnlocked: Boolean
  paliers: [Palier]
}
```

```

type World {
  name: String!
  logo: String
  money: Float
  score: Float
  totalangels: Int
  activeangels: Int
  angelbonus: Int
  lastupdate: String
  products: [Product]
  allunlocks: [Palier]
  upgrades: [Palier]
  angelupgrades: [Palier]
  managers: [Palier]
}

type Query {
  getWorld: World!
}

type Mutation {
  acheterQtProduit(id: Int!, quantite: Int!): Product
  lancerProductionProduit(id: Int!): Product
  engagerManager(name: String!): Palier
  acheterCashUpgrade(name: String!): Palier
  acheterAngelUpgrade(name: String!): Palier
  resetWorld: World!
}

```

Ce schéma est composé d'une entité World qui décrit le monde dans sa globalité avec ses propriétés, ses produits, les managers des produits, les upgrades achetables avec de l'argent ou des anges, et la liste des « allunlocks », cad les bonus débloqués une fois que **tous** les produits ont atteint une certaine quantité.

Il possède également une entité Product qui décrit les propriétés de chacun des produits, avec en particulier les liste des paliers de bonus débloqué une fois que **ce** produit a atteint une certaine quantité.

Ce type « Palier » mérite quelques explications. Il est en effet utilisé à la fois pour spécifier un *Manager*, un *Cash Upgrade*, un *Angel Upgrade*, et un bonus lié à la quantité d'un produit.

Voici comment on interprète sa valeur dans ces quatre situations :

- **Palier** dans la liste des managers :
 - name : nom du manager ;
 - logo : image du manager ;
 - seuil : somme nécessaire pour débloquent le manager ;
 - idcible : identifiant du produit géré par le manager ;
 - ratio, typeratio : non utilisés ;
 - unlocked : vrai ou faux selon que le manager est débloquent ;
- **palier** dans la liste des *Cash Upgrades* ou des *Angel Upgrades* :
 - name : nom de l'upgrade ;
 - logo : image de l'upgrade, on peut utiliser l'icône du produit auquel l'upgrade est lié, ou une icône spécifique si tous les produits sont concernés, ou encore une icône d'ange s'il s'agit d'un *Angel Upgrade* ;
 - seuil : Argent ou nombre d'anges nécessaire pour acheter l'upgrade ;
 - idcible : identifiant du produit ciblé par l'upgrade ou 0 si tous les produits, ou -1 si le bonus augmente l'efficacité des anges ;

- ratio :
 - Si typeratio est « vitesse », divise le temps de production par le ratio indiqué ;
 - Si typeratio est « gain », multiplie le revenu du produit par le ratio indiqué ;
 - Si typeratio est « ange », ajoute typeratio au bonus actuel des anges (par exemple si typeratio vaut 1, les anges gagnent 1% à leur bonus de production) ;
- typeratio : « vitesse », « gain » ou « ange » selon la cible du bonus ;
- unlocked : vrai ou faux selon que l'upgrade a été acheté ;
- **palier** dans la liste des « unlocks » lié à la quantité de produits :
 - name : nom de l'*unlock* ;
 - logo : image de l'*unlock*, on peut utiliser l'icône du produit auquel l'*unlock* est lié, ou une icône spécifique si c'est un *unlock* global ;
 - seuil : Quantité de produit à atteindre pour débloquent le bonus ;
 - idcible : identifiant du produit ciblé par le bonus ou 0 si tous les produits ;
 - ratio :
 - Si typeratio est « vitesse », divise le temps de production par le ratio indiqué ;
 - Si typeratio est « gain », multiplie le revenu du produit par le ratio indiqué ;
 - Si typeratio est « ange », ajoute typeratio au bonus des anges ;
 - typeratio : « vitesse », « gain » ou « ange » selon la cible du bonus ;
 - unlocked : vrai ou faux selon que l'*unlock* est débloquent.

Seul concession faite par rapport au jeu d'origine, nous ne spécifierons pas d'upgrades de type « ajout d'une certaine quantité de produits ». Le type d'upgrade sera donc uniquement vitesse, gain ou ange.

Mise en place de la partie serveur

Vous devez commencer par la partie backend, car c'est cette partie qui définit le monde et le frontend va donc en avoir besoin pour fonctionner

Reportez-vous à l'énoncé spécifique au backend, et faites ensemble la partie de mise en place qui consiste à définir les données de votre monde.

Une fois la mise en place réalisée, vous pouvez vous partager les rôles concernant le backend et le frontend. **Le reste de cet énoncé s'adresse donc au développeur de la partie frontend.**

Pour ne pas que la partie frontend puisse avancer indépendamment de la partie backend, un backend de test est mis à votre disposition à l'adresse :

<https://isiscapitalist.chl.connected-health.fr/graphql>

Vous pouvez vous en servir tant que votre partie backend n'implémente pas encore certains points d'accès nécessaire au frontend.

Mise en place du projet Angular

Reproduisez les mêmes opérations que pour la partie frontend de l'énoncé préparatoire (apéritif ISISCapitalist) pour créer la partie frontend du projet.

Pour générer vos requêtes GraphQL, aidez-vous du playground du serveur GraphQL pour les construire. Commencez par la requête qui ramène l'ensemble du monde. Elle est de la forme :

```
query getWorld($user: String!) {
  getWorld(user: $user) {
    name
    angelbonus
    logo
    money
    score
  }
}
```

```

totalangels
lastupdate
products {
  timeleft
  quantite
  vitesse
  revenu
  croissance
  cout
  logo
  name
  id
  ...

```

Placez-là dans le fichier queries.graphql et utilisez codegen pour générer le fichier operations.ts. Il faudra régulièrement revenir dans ce fichier queries.graphql pour y ajouter les nouvelles requêtes et mutations, au fur et à mesure que vous avancerez dans l'énoncé.

Description du travail attendu

Dans un premier temps, nous allons construire un client autonome qui ne synchronisera pas les actions de l'utilisateur avec le serveur. La seule interaction de ce client avec le serveur consistera à obtenir les données du monde lors du premier chargement de la page web.

En fonction de cette description, le client doit bâtir une mise en page présentant les éléments de base du jeu à savoir les différents produits (icône, nom, quantité, barre de production) et les éléments d'interaction (bouton d'achat des produits, boutons cliquables permettant de spécifier les quantités d'achat et de faire apparaître les différents *upgrades*, *unlocks* et autres *managers*) La Figure 7 présente un exemple de l'interface à obtenir.

Essentiellement cette interface repose sur une mise en page divisant la page en trois parties :

- Un bandeau d'entête annonçant le monde (icône et nom), l'argent du joueur, le bouton de quantité d'achat et un champ texte permettant au joueur de saisir son nom ;
- Un bandeau gauche spécifiant les boutons cliquables permettant d'afficher des fenêtres supplémentaires décrivant des éléments supplémentaires du jeu ;
- Une partie centrale listant les produits et leurs éléments d'interaction.

Category	Item Icon	Count	Price	Time/Value
Unlocks	Shopping Bag	1	4.00	00:00:50
Cash Upgrades	Recycling Bin	0	60.00	
Angel Upgrades	Wind Turbine	0	103680	
Managers	Bicycle	0	720.00	
Investors	Solar Panel	0	1.244 10 ⁶	

Figure 7 – Illustration de l'interface du jeu

Les produits sont eux-mêmes affichés sous la forme d'une mise en page en plusieurs parties :

- Une partie gauche présentant l'image du produit à laquelle se superpose sa quantité
- Une partie droite divisée en :
 - Une partie haute présentant la barre de progression de la production qui indique aussi le gain qui sera généré. Ne vous occupez pas pour l'instant du rendu de cette barre de progression, vous réserverez simplement l'espace qui lui sera nécessaire.
 - Une partie basse qui permet d'acheter plus de ce produit et qui indique également la quantité qui sera achetée et le coût associé, avec à côté le temps qui reste à s'écouler pour que la production du produit soit complète.

Pour styler l'affichage, on peut utiliser « Angular Material » qui est un ensemble de styles CSS et de widget. Pour installer ce composant, tapez la commande suivante à la racine de votre projet :

```
ng add @angular/material
```

Pour réaliser une mise en forme sous forme de grille, on peut utiliser le CSS3 Grid Layout qui permet facilement de spécifier des mises en page adaptatives en colonnes (voir par exemple https://developer.mozilla.org/fr/docs/Web/CSS/CSS_Grid_Layout).

Voici par exemple comment on peut définir un affichage avec un entête constitué de quatre colonnes, et une partie principale composée de deux colonnes, la deuxième colonne étant elle-même composée de cellules réparties sur deux colonnes

```
<div class="header">
  <div> logo monde </div>
  <div> argent </div>
  <div> multiplicateur </div>
  <div> ID du joueur </div>
</div>
<div class="main">
  <div> liste des boutons de menu </div>
  <div class="product">
    <div> premier produit </div>
    <div> second produit </div>
    <div> troisième produit </div>
    <div> quatrième produit </div>
    <div> cinquième produit </div>
    <div> sixième produit </div>
  </div>
</div>
```

Avec le style CSS suivant :

```
.header {
  display: grid;
  grid-template-columns: 25% 25% 30% 20%;
}

.main {
  display: grid;
  grid-template-columns: 20% 80%;
}

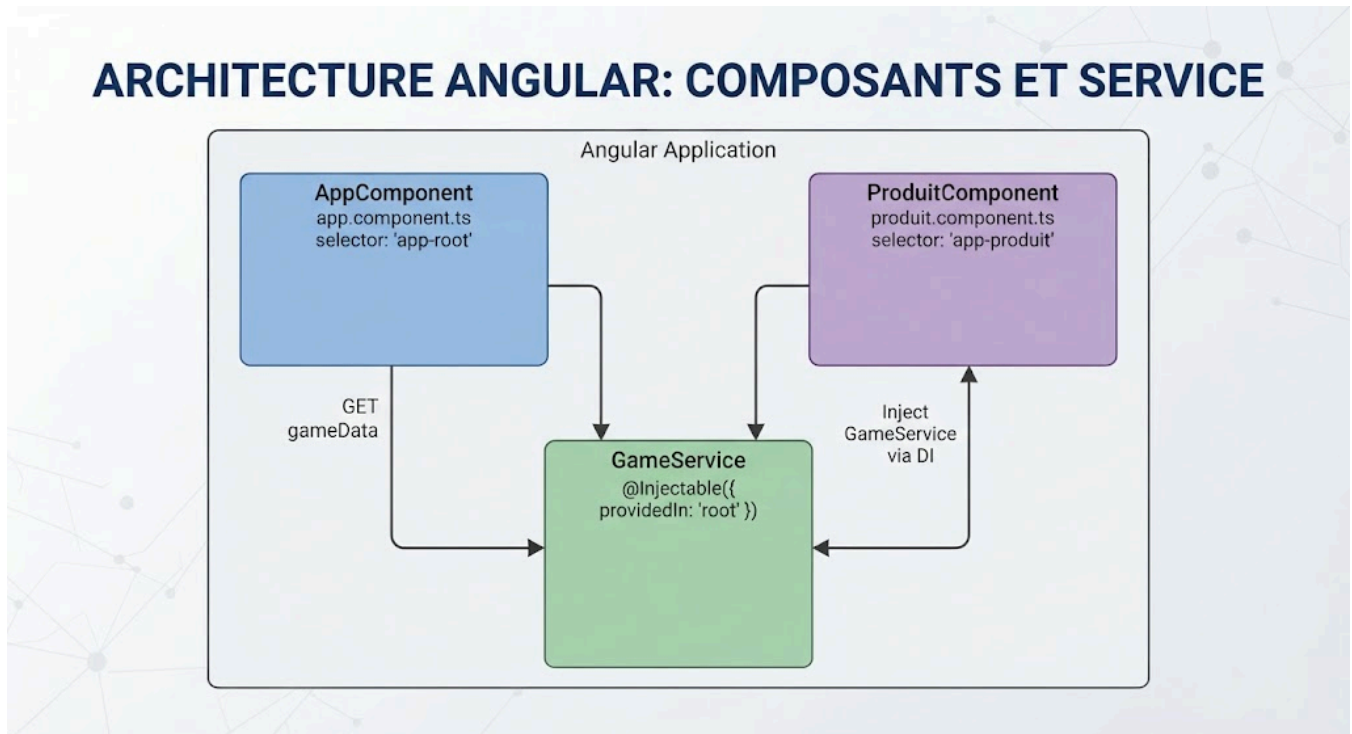
.product {
  display: grid;
  grid-template-columns: 1fr 1fr;
}
```

Vous adapterez cet exemple pour réaliser la mise en page que vous souhaitez pour votre jeu.

Création des premiers composants

En ce qui concerne l'organisation des sources dans le projet Angular, nous utiliserons deux composants et un service. Le premier, `app` (déjà créé lors de la phase d'initialisation du projet) s'occupera de définir tous les éléments globaux de l'IHM (icône et nom du monde, argent, multiplicateur d'achat, bandeau de droite avec les upgrades, etc.), mais délèguera à un second, `produit`, le soin de gérer l'affichage des éléments d'un produit.

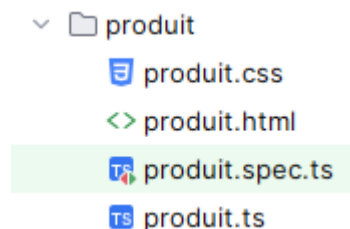
Enfin, un service `GameService` contiendra l'essentiel de la logique du code, ainsi que les signaux qui seront partagés entre `app` et `produit`.



Pour créer un nouveau composant automatiquement, on peut utiliser la commande :

```
ng g component produit
```

qui ajoute dans l'arborescence les fichiers nécessaires au composant product. Vous devez les voir apparaître dans Visual Studio Code.



Pour créer un service automatiquement, on peut utiliser la commande :

```
ng g service GameService
```

Nous allons doter le composant `GameService` du code nécessaire à faire un appel à la méthode `Getworld` de notre service web graphql. Pour cela nous devons représenter le nom du joueur et l'adresse du serveur sous la forme d'un signal :

```
user = signal('toto');
```

Définissez aussi un signal qui contient l'adresse de votre backend. Cela est nécessaire pour le chargement des images de vos produits (entre autres).

```
server = signal('http://localhost:3000');
```

Comme nous l'avons vu dans le TP apéritif, créez un signal correspondant à la requête `GetWorld(user)`. Le voici :

```
worldQuery = this.apollo.signal.query({
  query: GET_WORLD_QUERY,
  variables: () => ({ user: this.user() }),
});
```

Pour manipuler plus facilement ce signal, créez un signal dérivé qui va chercher directement sa partie `world` :

```
world = linkedSignal(() => this.worldQuery.data()?.getWorld);
```

Injectez le `GameService` dans le composant principal pour que celui-ci est aussi accès au monde :

```
@Component({
  imports: [BigvaluePipe, Produit, MatBadge, MatButtonModule, FormField],
  selector: 'app-root',
  styleUrls: ['./app.css'],
  templateUrl: './app.html',
})
export class App {
  // injection du service
  gameService = inject(GameService);
  // copie du signal dans la composant pour plus de facilité
  world = this.gameService.world;
}
```

Travaillons à présent sur le rendu visuel du monde.

Réalisation du *layout* général

Dotez les pages `app.html` et `app.css` du code HTML nécessaire pour spécifier les grandes lignes de votre mise en pages, en insérant aux endroits adéquats les valeurs associés aux propriétés de votre monde.

Par exemple, à l'endroit où doit s'afficher l'icône du monde, on pourra trouver le code html suivant :

```
<img class="round" [src]="gameService.server() + world()?.logo" />
```

Autre exemple, voici comment on peut insérer le nom du monde :

```
<span> {{ world()?.name }} </span>
```

Bien entendu, ces balises html devront être stylés pour réaliser un design un peu sympa (voir plus bas pour quelques éléments et astuces CSS).

Procédez ainsi pour tous les éléments généraux du monde (sauf pour le détail des produits).

Déléguez au composant `produit` le soin de réaliser le design d'un produit. Pour cela, à l'endroit où doit s'afficher par exemple le premier produit, vous insèrerez le code html suivant (les classes CSS sont à adapter en fonction du rendu que vous souhaitez obtenir) :

```
<div class="products">
  @for (j of [0, 1, 2, 3, 4, 5]; track $index) {
    <app-produit
      [prod]="world()?.products?.[j]" class="product">
    </app-produit>
  }
</div>
```

Dans le fichier `produit.ts`, vous récupérez le produit passé par le composant parent en définissant une propriété obtenue avec `input()`. Voici à titre exemple comment on peut le faire :

```
@Component({
  imports: [BigvaluePipe, SecondPipe],
  selector: 'app-produit',
  styleUrls: ['./produit.css'],
  templateUrl: './produit.html',
})
export class Produit {
  gameService = inject(GameService);
  // prod est un signal qui représente le produit
  prod = input<Product | undefined>();
}
```

A partir de ce point, vous disposez dans le composant `produit` d'une propriété *produit* qui correspond au produit à afficher.

Vous pouvez donc spécifier le code HTML du fichier `produit.html` qui réalise l'affichage d'un produit.

Éléments utiles de CSS

Voici quelques connaissances CSS qui vous permettront d'optimiser votre rendu graphique. Ces styles sont à placer soit dans le fichier CSS lié au composant dans lequel vous les utilisez.

Images arrondies :

Pour spécifier les images de vos produits (et plus tard de vos managers), vous pouvez d'une part spécifier leur taille et d'autre part les insérer dans des cercles en leur appliquant le style CSS suivant :

```
.round {
  width: 100px;
  height: 100px;
  border-radius: 50%
}
```

Superposition de deux éléments :

Pour superposer deux éléments, il faut les encapsuler ensemble dans une même balise `<div>` à laquelle on applique un positionnement relatif, et spécifier l'un des deux éléments (ou les deux) en positionnement absolu.

Ainsi si au lieu de mettre l'un sous l'autre les deux éléments « contenu 1 » et « contenu 2 », on veut légèrement les superposer, on pourra écrire :

```
.lesdeux {
  position: relative;
}

.le_second {
  position: absolute;
  // on met cet élément à 10 pixels du bas de son conteneur
  // du coup il se superpose à l'autre qui est dans le flux normal
  bottom: -10px;
}
```

```
<div class="lesdeux">
  <div class="le_premier">Contenu 1</div>
  <div class="le_second">Contenu 2</div>
</div>
```

Les pipes d'Angular

A plusieurs reprises vous serez amené à formater différentes valeurs dans votre interface. C'est par exemple le cas des grands nombres comme l'argent possédé par le joueur (qui doit s'afficher sous la forme de puissances de 10 quand ce nombre est important) ou le temps restant pour la production d'un produit qui doit s'afficher sous la forme *heure : minutes : secondes : dixièmes de secondes*.

Les *pipes* d'Angular fournissent une solution élégante à ce problème en permettant de définir un transformateur qui prend en paramètre une valeur (par exemple le temps en millisecondes) et qui produit une chaîne de caractères. A titre d'exemple définissons un pipe qui prend en paramètre un nombre flottant et qui produit son formatage en puissance de dix, avec quatre chiffres significatifs.

Pour créer le *pipe*, on peut utiliser la commande :

```
ng generate pipe bigvalue
```

La classe produite (`bigvalue-pipe.ts`) possède une méthode `transform` qu'il faut coder pour réaliser la transformation. Elle prend en paramètre la valeur à transformer et doit retourner la chaîne de caractère produite. Une implémentation un peu naïve de cette méthode pourrait être :

```
transform(valeur: number, args?: any): string {
  let res : string;
  if (valeur < 1000)
    res = valeur.toFixed(2);
  else if (valeur < 1000000)
    res = valeur.toFixed(0);
  else if (valeur >= 1000000) {
    res = valeur.toPrecision(4);
    res = res.replace(/e\+(.+)/, " 10<sup>$1</sup>");
  }
  return res;
}
```

Le code ci-dessus restreint la précision du grand nombre flottant à quatre chiffres, puis transforme l'exposant en un formatage adéquat pour un rendu HTML faisant apparaître un 10ⁿ. Cette implémentation peut éventuellement être améliorée en fonction de vos propres désirs d'affichage et de design.

Pour ensuite utiliser le *pipe* dans le rendu HTML, on peut par exemple écrire :

```
<span id="money" [innerHTML]="world()?.money ?? 0 | bigvalue"> </span>
```

Actions du joueur

Le *layout* étant en place, nous allons commencer à implémenter le traitement des actions du joueur. La première d'entre elle consiste à cliquer sur l'icône d'un produit pour en lancer la production.

Démarrage de la production d'un produit

Dans le fichier `produit.html`, ajoutez un gestionnaire d'évènement (*click*) sur les icônes de produits qui mène à une méthode `startFabrication()` que vous allez implémenter dans `produit.ts`.

La méthode `startFabrication()` doit démarrer une barre de progression dans l'espace prévu à cet effet. Pour le rendu de cette barre, vous pouvez tout simplement utiliser le code html suivant avec le style css associé :

```
<div class="progressbar">
  // progress() est un signal qui varie de 0 et 100 et qui donne le taux de progression de la barre
  <div class="fill" [style.width.%]="progress()"></div>
</div>
```

```
.progress {
  height: 50px;
  width: 100%;
}

.fill {
  height: 100%;
  background-color: #4caf50;
  width: 0%;
  transition: width 0s linear;
}
```

Faites maintenant en sorte qu'un clic sur l'icône du produit lance sa production. Pour l'instant laissez vide la méthode appelée (voir la suite pour savoir quoi mettre dedans).

La boucle principale de calcul du score

Nous devons à présent définir la boucle principale de mise à jour de la production de notre produit. Cette méthode sera appelée à intervalle régulier (par exemple tous les dixièmes de seconde) pour mettre à jour la barre de progression et calculer l'évolution de l'argent gagné en fonction du temps qui passe.

En javascript, c'est la méthode `setInterval(...)` qui permet d'exécuter du code toutes les *ms* millisecondes. Ainsi si on appelle `calcScore()` notre fonction de calcul du score, on peut, lors du démarrage du composant (méthode `ngOnInit()`), l'appeler tous les dixièmes de seconde en écrivant :

```
ngOnInit() {
  setInterval(() => {
    this.calcScore();
  }, 100);
}
```

Il ne reste plus qu'à implémenter une méthode `calcScore()` qui, pour un produit et en fonction du temps écoulé depuis la dernière fois, calcule combien de fois le produit a été produit (tant que le manager du produit n'est pas débloqué, on ne pourra pas produire plus d'un produit par clic). Cette fonction est la même que celle que le backend doit aussi mettre en œuvre. On vous recommande donc de réfléchir ensemble à cette fonction et d'utiliser la même côté front et côté back.

Il faudra probablement que le composant `Produit` manipule des compteurs de progression de la production sous la forme de signaux. Il vous faudra un signal `timeLeft` pour savoir où en est la production du produit, et repérer les dates de mise à jour en appelant la méthode `performance.now()` qui vous donne l'heure avec une précision d'une milliseconde.

Il faudra également lancer la barre de production et la remettre à zéro à la fin de la production. On vous recommande d'utiliser la méthode `requestAnimationFrame(this.animate)` à laquelle on passe une méthode qui anime la barre de progression.

Typiquement cette méthode va ressembler à :

```
animate = (currentTime: number) => {
  const elapsed = currentTime - this.starttime;
  const progressPercent = Math.min((elapsed / this.vitesse()) * 100, 100);

  this.progress.set(progressPercent);
  if (progressPercent < 100) {
    requestAnimationFrame(this.animate); // utiliser rAF pour de la fluidité à 60fps
  } else {
    // barre à 100%, on remet à 0 pour la prochaine production
    this.progress.set(0);
  }
};
```

Au final, chaque clic sur un produit lance sa production, et à la fin **vous devez augmenter le score du gain obtenu**.

Vous allez déléguer cette opération au `GameService` en le dotant d'une méthode et la méthode `productionDone(prod: Product, qt: number)` qui prend en paramètre le produit et sa quantité produite. Appelez cette méthode à chaque production, et rédigez son contenu pour augmenter l'argent possédé et le score du monde.

Voici par exemple comment écrire cette méthode :

```
productionDone(prod: Product, qt: number) {
  let gain = prod.revenu * prod.quantite * qt;
  // on met à jour le signal représentant le monde
  this.world.update((world) => {
    if (!world) return world;
    // On retourne le nouveau monde, copie du précédent avec mise à jour
    // des propriétés money et score
    return { ...world, money: world.money + gain, score: world.score + gain };
  });
}
```

L'achat de produit

Le joueur doit avoir la possibilité d'acheter une certaine quantité de produit en cliquant sur le bouton prévu à cet effet. Le nombre de produits achetable dépend d'une part de l'argent qu'il possède, d'autre part du commutateur général `x1`, `x10`, `x100`, ou `xMax` situé en haut à droite de l'interface.

Commencez-donc par implémenter ce bouton commutateur qui est géré par le `app` et qui doit fonctionner de la façon suivante :

- Chaque clic sur le bouton doit lui faire changer de position selon le cycle `x1 -> x10 -> x100 -> Max -> x1`, etc.
- A chaque changement de position, le composant doit prévenir chaque produit de la nouvelle position ;
- Les produits doivent modifier le marqueur de quantité d'achat selon la nouvelle position. Quand il s'agit des positions `x1`, `x10` ou `x100`, on prendra soin de rendre le bouton d'achat cliquable uniquement si le joueur est en capacité financière d'acheter la quantité spécifiée. Cependant, quand le bouton commutateur est sur la position `Max`, il s'agit de calculer la quantité maximale achetable par le joueur de ce produit, et d'inscrire cette quantité dans le bouton d'achat.

Quelques indications pour réaliser cela :

- Dans le composant `app.ts`, on utilisera un signal (appelons le `qtmulti`) pour stocker l'état actuel du commutateur d'achat général.
- On transmettra la valeur de cette propriété aux composants produit, par exemple en ajoutant à la balise `<app-produit>` l'attribut `[qtmulti]` comme ceci :

```
<app-produit
  [prod]="world()?.products?.[j]"
  [qtmulti]="qtmulti()"
  class="product">
</app-produit>
```

- Dans `produit.ts`, on déclarera un signal `maxCanBuy()` qui calcule la quantité supplémentaire maximale achetable par le joueur de ce produit. Notez que chaque achat d'un produit supplémentaire coûte le prix actuel du produit multiplié par son pourcentage de croissance. Ainsi si x est le coût actuel d'un exemplaire de produit, et si c est la croissance de ce coût, alors acheter un produit de plus coûtera $x * c$, acheter deux produits coûtera $x * c + x * c * c$, trois produits $x * c + x * c * c + x * c * c * c + x * c * c * c * c$, etc. Pour généraliser, acheter n produits demandera

$$\text{coût} = x * (1 + c + c^2 + c^3 + \dots + c^n)$$

A vous d'être capable à partir d'une certaine somme possédée par le joueur, de trouver n .

- On déclarera également un signal `numberToBuy` qui en fonction du signal `qtratio` calcule combien on peut acheter d'exemplaires du produit (1, 10, 100, où le max)
- Un dernier signal `canBuy` définira un booléen qui détermine si on peut acheter la quantité demandée par le signal `numberToBuy`. Ce signal sera utilisé pour activer ou désactiver le bouton d'achat.
- Enfin, l'achat du produit a pour conséquence la modification du monde puisqu'il faut augmenter la quantité du produit, mettre à jour le coût du prochain produit, et bien entendu décrémenter de l'argent du monde le coût d'achat. Toutes ces opérations seront réalisées dans `GameService` dans une méthode `buyProduct(qt: number, product: Product)` dont la forme va ressembler à ceci :

```
async BuyProduct(qt: number, product: Product) {
  let world = this.world();
  if (!world) return;
  // on génère une nouvelle liste de produits en transformant chaque produit (map)
  let newproductlist = world.products.map((p) => {
    // si ce n'est pas le produit passé en paramètre, on ne change rien au produit
    if (p.id !== product.id) return p;
    else
      // sinon on transforme l'ancien produit en mettant à jour ses propriétés qui ont changé
      return {
        ...p,
        // nouvelle quantité
        quantite: p.quantite + qt,
        // nouveau cout du prochain produit
        cout: p.cout * Math.pow(p.croissance, qt),
      };
  });
  // nouvelle version du monde avec les certains produits changés
  let newworld = { ...world, products: newproductlist };
  let cost = /* calcul du cout d'achat */

  // on termine par la mise à jour du signal du monde
  this.world.update((world) => {
    if (!world) return world;
    return { ...newworld, money: world.money - cost };
  });
  // appeler la mutation graphql qui prévient le backend de l'achat
  this.acheterProduitsGraphQL(product.id, qt);
}
```

Cette méthode se termine par l'appel graphql vers le serveur pour lui transmettre l'achat du joueur.

Pour implémenter cette méthode vous devez faire deux choses:

- Ajouter dans le fichier `queries.graphql` la mutation concernée à savoir :

```
mutation acheterQtProduit($user: String!, $id: Int!, $quantite: Int!) {
  acheterQtProduit(user: $user, id: $id, quantite: $quantite) {
    id
  }
}
```

- Régénérez le fichier `operation.ts` en utilisant en ligne de commande la commande `npm run codegen`
- Ajoutez dans le fichier `GraphService` le code définissant la mutation et l'appelant avec un signal comme ceci :

```
readonly acheterProduitsMutation = this.apollo.signal.mutation(ACHERER_QT_PRODUIT_MUTATION);

async acheterProduitsGraphQL(id: number, qt: number) {
  try {
    const { data } = await this.acheterProduitsMutation.mutate({
      variables: { user: this.user(), id, quantite: qt },
    });
  } catch (error) {}
}
```

Vous pouvez tester la valeur du signal `acheterProduitsGraphQL` si vous avez besoin de savoir si la requête a réussi ou pas. Par exemple, vous pouvez afficher dans `produit.ts` un `toast` d'erreur avec le code suivant :

```
if (this.gameService.acheterProduitsMutation.error()) {
  this._snackBar.open("Erreur de transmission serveur pour l'achat du produit", 'ok', {
    duration: 2000,
  });
}
```

Maintenant que vous savez faire des mutations GraphQL envoyés au serveur, profitez-en pour créer la mutation qui signale au serveur la mise en production manuelle d'un produit (sauf si vous l'avez déjà fait). Cette mutation doit être envoyée par `GraphService` à chaque fois qu'un joueur lance la production d'un produit non encore automatisé.

Voici le code graphql de cette mutation, procédez comme pour la mutation d'achat pour l'ajouter dans `GraphService` :

```
mutation lancerProductionProduit($user: String!, $id: Int!) {
  lancerProductionProduit(user: $user, id: $id) {
    id
  }
}
```

Les managers

Interface pour lister les managers

Dans cette partie nous allons implémenter la partie cliente de l'achat de managers qui permettront d'automatiser la production de produits.

La liste des managers doit apparaître lorsque le joueur clique sur le bouton « *managers* » situé dans la colonne gauche de l'interface. Cette liste doit venir se superposer à l'interface en cours et elle devra se fermer quand le joueur cliquera sur son bouton de fermeture.

Une façon simple de gérer des fenêtres superposées est d'utiliser un style css qui positionne la section en fixe. Par exemple :

```
.modal {
  position: fixed; /* reste en place */
  z-index: 2; /* au dessus du reste */
  width: 800px; /* largeur */
  height: 800px; /* hauteur */
  overflow: auto; /* barre de scroll si besoin */
  background-color: white; /* opaque */
  border: solid;
  top:10%; /* a 10% du haut */
  left:20%; /* a 20% de la gauche */
}
```

Une fois cela fait, on peut ajouter dans le `app.html` la partie destinée à être affichée sur demande, avec un `@if(...)` qui vérifie s'il faut l'afficher ou pas. Voici un exemple de code qui réalise l'affichage des managers :

```
// le signal showManagers() indique si on veut afficher la liste ou pas
@if (showManagers()) {
  <div class="modal">
    <div>
      <h1 class="title">Managers make you feel better !</h1>
    </div>
    <div>
      // on cycle sur la liste des managers du monde
      @for (manager of world()?.managers; track manager.name) {
        // on affiche que les managers pas encore débloqués
        @if (!manager.unlocked) {
          <div class="managerslayout">
            <div>
              <div class="logo">
                <img class="round" [attr.src]="gameService.server() + manager?.logo" />
              </div>
            </div>
            <div>
              <div class="managername">{{ manager.name }}</div>
              <div class="managercible">
                {{ world()?.products?.[manager.idcible - 1]?.name }}
              </div>
              <div class="managercost">{{ manager.seuil }}</div>
            </div>
            <div>
              <div
                class="hirebutton"
                // engagement du manager
                (click)="this.gameService.hireManager(manager)"
                // on ne rend achetable le manager que si on a assez d'argent
                [class]="{ selectable: (world()?.money ?? 0) >= manager.seuil }">
                Hire !
              </div>
            </div>
          </div>
        }
      }
    </div>
    <button
      class="closebutton"
      mat-raised-button
      color="primary"
      // quand on clique sur close, on inverse la valeur du signal showManagers
      (click)="showManagers.update((u) => !u)">
      Close
    </button>
  </div>
}
```

```

    </button>
  </div>
</div>
}

```

Utilisez cette façon de faire pour compléter le fichier `app.html` pour qu'il remplisse la fenêtre des managers en fonction de ceux définis dans votre monde selon le modèle proposé Figure 8 (mais vous pouvez choisir un autre design si vous préférez)

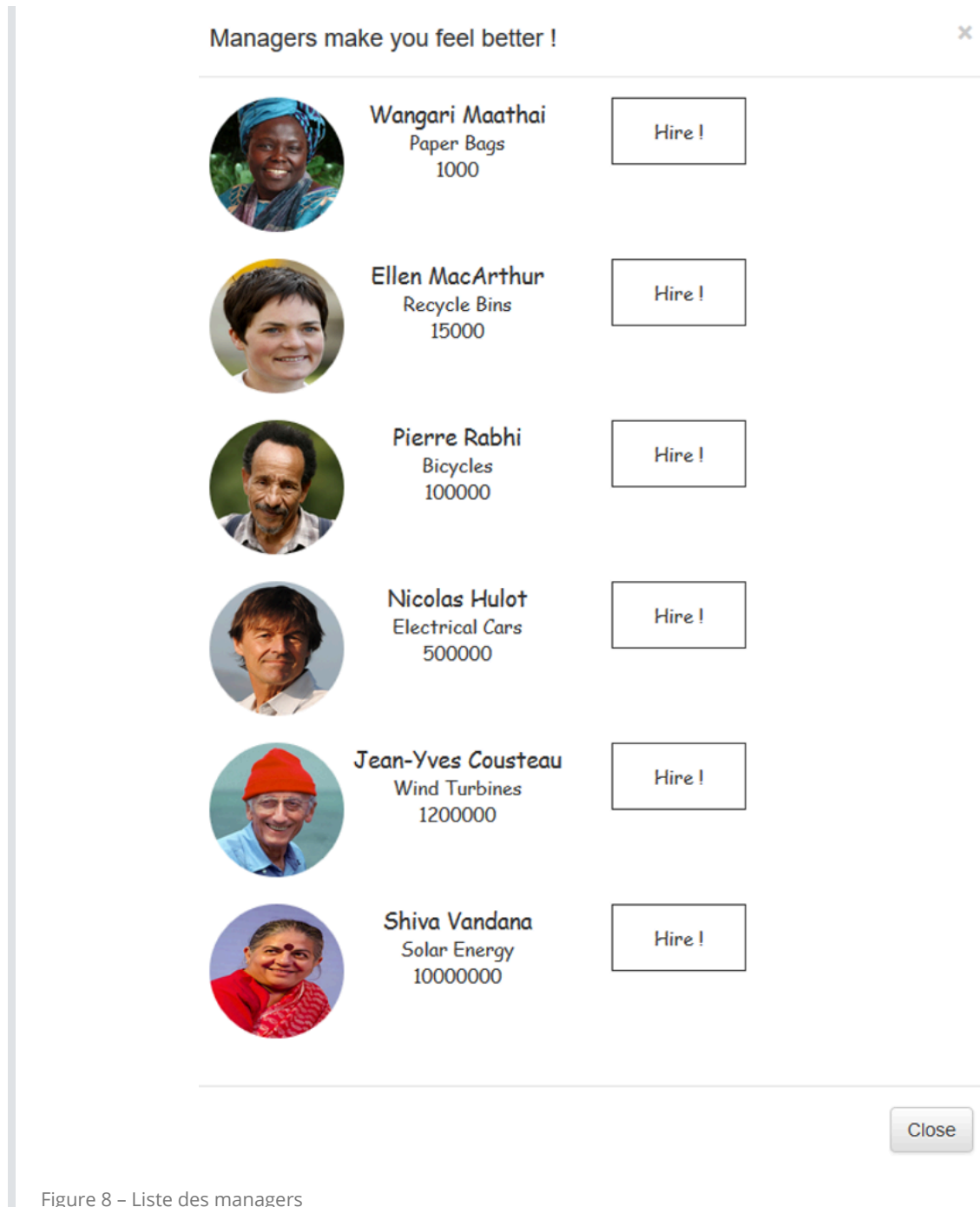


Figure 8 – Liste des managers

Engagement d'un manager

Implémentez le gestionnaire d'évènement associé au `click` sur le bouton d'engagement d'un manager. Ce gestionnaire doit être implémenté par dans `GraphService` et :

- Vérifier que l'argent du joueur est suffisant pour acheter le manager en question.
- Retirer le coût du manager de l'argent possédé par le joueur.
- Positionner la propriété `unlocked` du manager à vrai. Pareillement pour la propriété `managerUnlocked` du produit lié à ce manager.

- Vérifiez que les fonctions `calcScore()` et `animate` du composant `Produit` lance immédiatement la production d'un produit dont le manager est débloqué, même s'il n'était pas déjà en production.

Afficher un message éphémère pour l'utilisateur

Nous voulons qu'un message d'information soit affiché au joueur lorsque qu'il vient d'engager un nouveau manager. Nous aurons aussi besoin de ce genre de feedback lorsque que le joueur débloquera des *unlocks* ou des *upgrades*, ou encore pour prévenir le joueur d'une mauvaise transmission d'informations entre le client et le serveur.

La méthode recommandée est d'utiliser pour cela le composant snack-bar d'Angular Material. L'idée simple est de déclarer un signal `snackmessage` dans `GameService` et de faire en sorte que tout changement de valeur de ce signal déclenche l'affichage d'un *toast* dans l'interface du composant principal. On obtient cela en déclarant un *effet* dans le constructeur de `app.ts` comme ceci :

```
constructor() {
  effect(async () => {
    this._snackBar.open(this.gameService.snackmessage(), 'ok', { duration: 2000 });
  });
}
```

Cela fonctionne car l'instruction `effect` re-exécute son code en cas de changement de la valeur d'un des messages contenus dans son code.

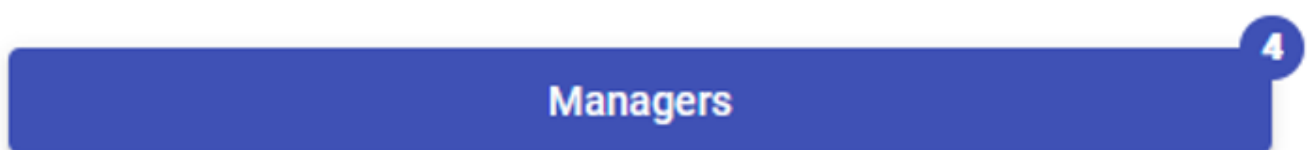
Badger les boutons pour informer le joueur

« Angular Material » permet de « badger » certains éléments de l'interface de façon à modifier légèrement leur apparence. Quand il s'agit d'un bouton cliquable, ce visuel permet d'avertir le joueur qu'une action est disponible en cliquant sur ce bouton.

Par exemple voici le bouton d'accès à la liste des managers sans badge :



Et voici le même bouton badgé avec le nombre de managers achetable :



Pour « badger » un bouton, on lui ajoute un attribut pour donner la valeur du badge, et un autre pour déterminer si le badge est visible ou pas. Par exemple, si on déclare un signal `badgeManagers` dans `app.ts` avec pour valeur le nombre de managers actuellement achetable par le joueur, le code suivant place un badge sur le bouton « Managers » quand un achat est possible :

```
<button
  class="actionb"
  mat-raised-button
  color="primary"
  [matBadge]="badgeManagers()"
  [matBadgeHidden]="badgeManagers() == 0"
  (click)="showManagers.update((u) => !u)">
  Managers
</button>
```

Bien entendu, le signal `badgeMangers` doit être un signal calculé en fonction de l'argent possédé et des seuils de la liste des managers pas encore débloqués.

Prévenir le backend

Comme pour les autres opérations du joueur, l'achat du manager doit être envoyé au backend. Voici la mutation :

```
mutation EngagerManager($user: String!, $name: String!) {
  engagerManager(user: $user, name: $name) {
    name
  }
}
```

Procédez comme pour les autres mutations pour l'appeler dans `GraphService` lors de engagement d'un manager.

Le nom du joueur

Pour l'instant nous avons joué avec un nom de joueur spécifié en dur dans le code, plus précisément dans le signal `user` de `GraphService`. Nous voulons afficher sur notre interface, un champ de formulaire permettant de préciser et de changer ce nom. Nous aimerions aussi que le changement du nom, commute la partie sur celle de ce joueur.

Ajoutez à votre interface un champ de formulaire de type texte (en haut à droite sur la Figure 7) permettant à l'utilisateur de spécifier un pseudo. A coté de ce champ texte, implémentez un bouton *Refresh* qui permettra de rafraichir le monde à partir du backend.

Dans le fichier `GraphService` ajoutez le code permettant de récupérer la valeur du champ texte sous la forme d'un signal. Le voici :

```
interface UserLogin {
  name: string;
}

@Service()
export class GameService {

  UserloginModel = signal<UserLogin>({
    name: '',
  });

  loginForm = form(this.UserloginModel);
}
```

Dans le formulaire html, liez le champ de formulaire au signal `loginForm` et ajoutez un gestionnaire d'évènement qui appelle une méthode une fois que l'utilisateur valide le nom en appuyant sur la touche entrée :

```
<div>
  <div>Your ID :</div>
  <input type="text" [formField]="gameService.loginForm.name" (keydown.enter)="gameService.commitName()"
/>
  <input type="button" value="Refresh" (click)="gameService.refreshworld()" /></div>
</div>
```

Et ajoutez la méthode `commitName()` pour positionner le signal `user` à la valeur saisie :

```
commitName() {
  let field = this.loginForm.name().value();
  localStorage.setItem('username', field);
  this.user.set(field);
}
```

Vous remarquer l'appel à `localStorage.setItem('username')` qui permet d'écrire dans le stockage local du navigateur, le nom saisi.

Il ne reste plus, en cas de rechargement de la page, qu'à charger ce nom stocké en local comme valeur initiale du champ du formulaire. On fait cela dans le constructeur de `GameService` comme ceci :

```
constructor() {
  // on regarde s'il existe un username stocké en local
  let username = localStorage.getItem('username');
  // bonus, si pas de username, on génère un username aléatoire au lieu de le laisser vide
  if (!username || username === '') {
    username = 'Captain' + Math.floor(Math.random() * 10000);
  }
  // on positionne ce username comme valeur initiale du formulaire et du user
  this.loginForm.name().value.set(username);
  this.user.set(username);
}
```

Enfin, pour le refresh du monde, il suffit d'appeler la méthode `refresh` sur le signal graphql générant la requête vers le monde comme ceci :

```
refreshWorld() {
  this.worldquery.refetch();
}
```

Les *unlocks*

Affichage des *unlocks*

Ajoutez le code nécessaire à l'affichage des *unlocks* attachés à chaque produit. La réalisation de cette partie ressemble énormément à la gestion des managers. La seule différence réside dans le fait que les *unlocks* ne s'achètent pas, ils sont automatiquement débloqués quand la quantité de produits atteint le seuil qui leur est attaché. Le bonus associé aux *unlocks* peut être de type vitesse ou gain et l'affichage doit clairement faire apparaître le type et la quantité de bonus obtenu.

Tout comme pour les managers, les *unlocks* déjà débloqués ne doivent pas être affichés.

Concevez donc une fenêtre de type *modal* qui sera ouverte lors d'un clic sur le bouton *unlock*. La fenêtre doit ressembler à celle affichée Figure 9. Si les *unlocks* sont trop nombreux, vous pouvez choisir de n'afficher que les *n* premiers, ou de n'afficher que le prochain seuil associé à chaque produit.

Want to maximize profits ? Get your investments to these quotas ! ✕



Don't forget your paper bag !

75

Paper Bags VITESSE x2



Give me some good bins !

20

Recycle Bins VITESSE x2



More Bicycles !

20

Bicycles VITESSE x2



These cars are wizzzzzz !

20

Electrical Cars VITESSE
x2



I feel like the wind !

20

Wind Turbines VITESSE
x2

Figure 9 – Liste des *unlocks*

Prise en compte des *unlocks* par le client

Adaptez le code du client pour prendre en compte les seuils atteints. Pour réaliser cela, vous devez vérifier, à chaque nouvelle quantité de produit acheté, si un seuil spécifié par un *unlock* a été atteint.

Il y a deux sortes d'*unlocks* :

- Ceux qui sont spécifiques à un produit et qui se déclenchent quand ce produit a atteint une certaine quantité. Ceux-là doivent être vérifiés par le composant `product.component.ts` à chaque augmentation de la quantité du produit.
- Ceux qui se déclenchent quand **tous** les produits ont atteint une certaine quantité (les *allunlocks*). Ceux-là doivent être vérifiés par le composant `app.component.ts` dès qu'un des produits voit sa quantité augmentée. La difficulté c'est que la prise en compte de ces *allunlocks* doit se faire au niveau de tous les produits et immédiatement. Il va donc falloir trouver un

moyen pour le parent puisse parcourir la liste de ses enfants pour leur faire prendre en compte les *unlocks* généraux venant de se débloquent (voir ci-après).

Dans tous les cas, l'application d'un *unlock* à un produit consiste :

- S'il s'agit d'un *boost* de revenu, il suffit de multiplier le revenu du produit par le bonus obtenu.
- S'il s'agit d'un *boost* de vitesse, il s'agit de diviser la vitesse du produit par le bonus obtenu. Si le produit est déjà en production, cela doit se traduire par une accélération de sa barre de progression

Enfin quand un bonus de seuil est obtenu, prévenez le joueur en envoyant un message éphémère sur l'interface comme vous l'avez fait en cas d'achat d'un nouveau manager :

✓ Paper Bags Speed x2

Les Cash upgrades

Affichage des upgrades

Tout comme vous l'avez fait pour les managers, concevez une fenêtre modale présentant les prochains *upgrades* disponibles en cas de clic sur le bouton « Cash Upgrades ».

Nous ne donnerons pas plus d'explications ici, le travail à faire étant très proche de celui réalisé pour l'affichage des managers et des *unlocks*. Notez simplement que les *upgrades* doivent pouvoir être achetés par le joueur, il faudra donc prévoir un bouton prévu à cet effet comme l'illustre la Figure 10. Là encore, n'hésitez pas à n'afficher que les n premiers *upgrades* si la liste est trop longue.

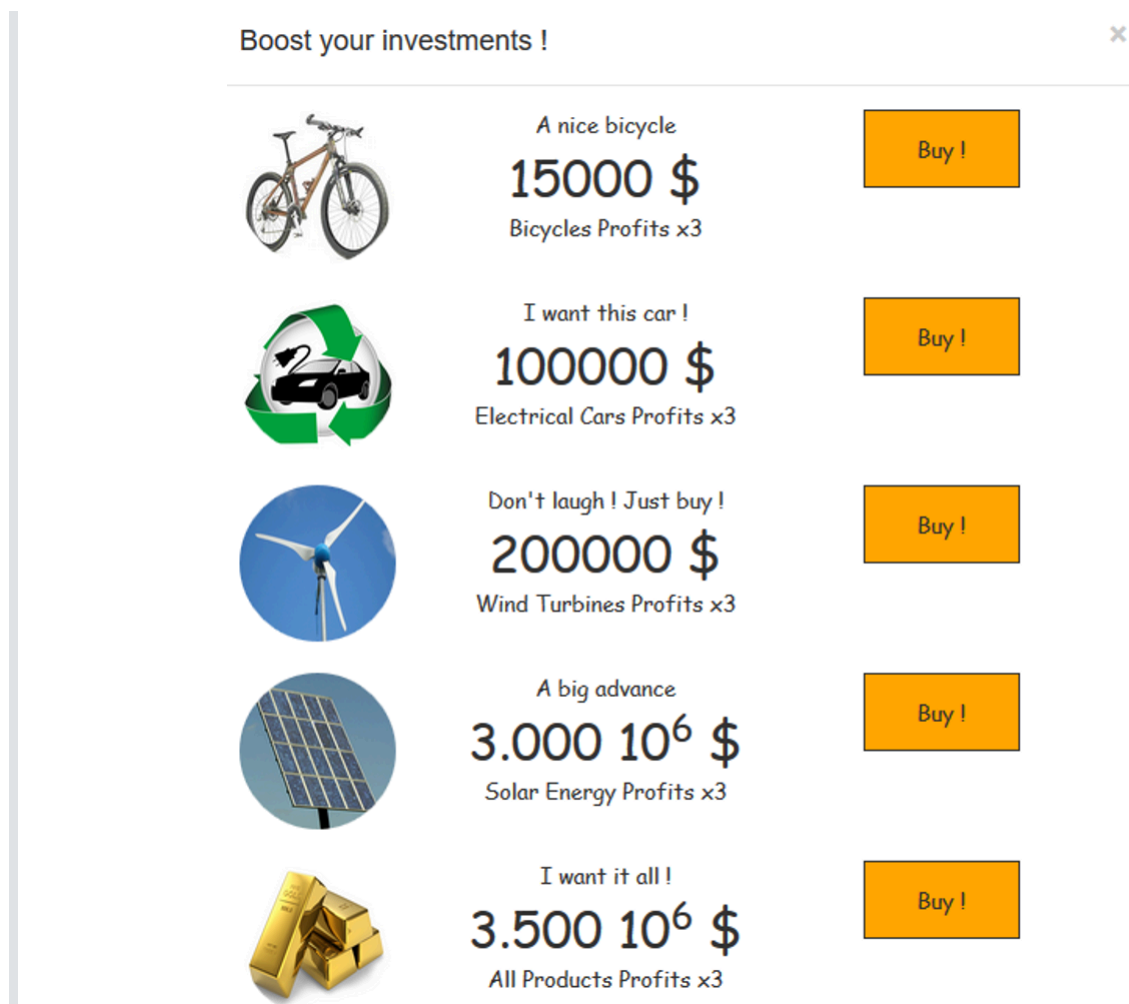
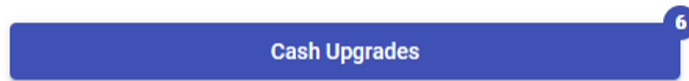


Figure 10 – Liste des *upgrades*.

Prévoyez également un badge qui viendra apparaître sur le bouton « upgrades » quand le joueur possèdera la somme nécessaire pour acheter au moins un des upgrades non encore débloqués.



Prise en compte des *upgrades* par le client.

Ajoutez dans `GraphService` le code pour appliquer aux produits concernés le bonus obtenu en cas d'achat d'un *upgrade*. Comme pour les *unlocks*, le bonus peut être de type *gain* ou *vitesse* (il peut sans doute être aussi de type *ange* mais nous n'avons pas encore implémenté les anges à cet endroit de l'énoncé).

L'application des bonus dus aux upgrades doit être en tout point identique à l'application des bonus dus aux *unlocks*. Une grande partie du code doit donc être repris de la partie précédente. Notez que tout comme pour les *allunlocks*, certains upgrades s'appliquent à tous les produits.

N'oubliez pas de transmettre l'achat de l'upgrade au backend avec la mutation :

```
mutation AcheterCashUpgrade($user: String!, $name: String!) {
  acheterCashUpgrade(user: $user, name: $name) {
    name
  }
}
```

Gestion des anges

Comme nous l'avons précisé lors de la description du *gameplay*, le joueur a la possibilité d'accumuler des anges pendant la partie. Le nombre d'ange gagné dépend de l'argent accumulé par le joueur. On parle ici de ses revenus totaux, pas de son argent actuel. C'est la raison pour laquelle le monde possède une propriété `score` qui représente l'argent total gagné par le joueur depuis le début de la partie. Le nombre d'ange gagné dépend donc directement de ce score.

Les anges n'ont aucun effet tant que le joueur ne remet pas la partie à zéro. Quand cela se produit, les anges gagnés commencent alors à procurer un bonus de 2% par ange aux revenus. Certains de ces anges peuvent également être dépensés en Angel Upgrade pour procurer des bonus supplémentaires **en remplacement** du bonus de 2% qu'ils apportaient.

C'est pourquoi le monde possède également une propriété `totalangels` qui représente les anges accumulés depuis le début de la partie, mais également une propriété `activeangels` qui représente les anges actuellement actifs. La différence entre ses deux propriétés représente les anges dépensés en *angel upgrades*.

Ainsi le nombre d'anges **supplémentaires** gagnés par la partie en cours est donc égal à :

$$\text{nombre d'anges} = 150 \times \sqrt{\frac{\text{gains cumulés}}{10^{15}}} - \text{totalangels}$$

Implémentez le clic sur le bouton « Investors ». Ce bouton doit afficher une fenêtre donnant le nombre d'anges actuellement actifs, ainsi que le nombre d'anges supplémentaires accumulés par la partie en cours. Cette fenêtre doit également proposer un bouton permettant de remettre à zéro la partie et donc de récupérer les anges supplémentaires. La Figure 11 illustre à quoi doit ressembler cette fenêtre.

En cas de click sur le bouton de « reset », le client doit prévenir le serveur en utilisant la mutation suivante :

```
mutation ResetWorld($user: String!) {
  resetWorld(user: $user) {
    name
  }
}
```

Suite à cette requête, le client doit remettre le monde dans l'état initial. Le plus simple pour réaliser cela est de demander le *refresh* du monde via la méthode `refreshworld()` du `GameService`.

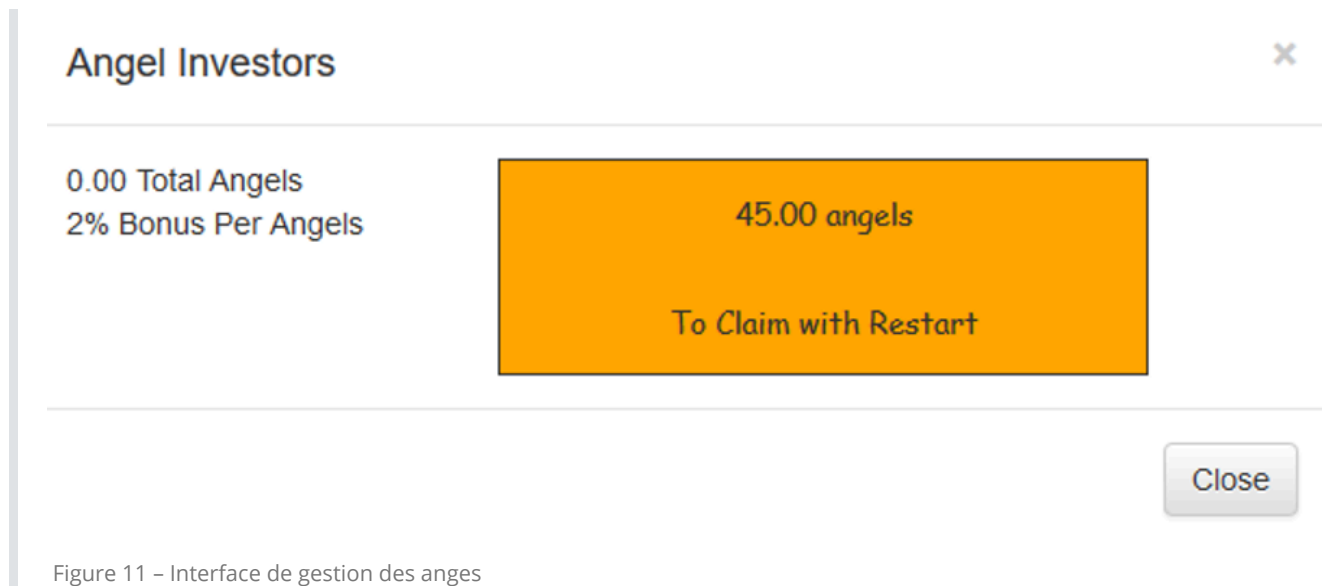


Figure 11 – Interface de gestion des anges

Prise en compte des anges dans les gains

Enfin il reste à modifier les fonctions de calcul du score, pour qu'elles appliquent le bonus de revenu par ange actif. Ce bonus pouvant évoluer, on le trouvera dans la propriété `angelbonus` du monde (initialement il est fixé à 2%).

Le revenu gagné par la production d'un produit est alors de :

$$product.quantite * product.revenu * (1 + world.activeangels * world.angelbonus / 100)$$

Gestion des *Angel Upgrades*

Il ne reste plus qu'à mettre en place l'interface d'achat des *Angel Upgrades* comme illustré par la Figure 12.

Le badge du bouton « Angel Upgrades » sera activé quand le joueur aura accumulé le nombre d'anges nécessaires à l'achat d'au moins 1 *upgrade*.

Spend your Angels Wisely !



	Angel Sacrifice 10.00 angels All Products Profits x3	Buy !
	Angelic Mutiny 100000 angels Angel Effectiveness + 2%	Buy !
	Angelic Rebellion 1.000 10⁸ angels Angel Effectiveness + 2%	Buy !
	Angelic Selection 1.000 10⁹ angels All Products Profits x5	Buy !

Figure 12 - Liste des Angel Upgrades

Prise en compte des *angel upgrades* par le client.

Lors de l'achat d'un *angel upgrade*, le client doit appliquer l'upgrade sur les anges ou les produits concernés, selon le type de cet upgrade.

S'il s'agit d'un upgrade de type « ange », alors on augmentera le bonus de production apporté par les anges selon la quantité de bonus de l'upgrade.

Sinon on réutilisera le code déjà en place pour appliquer soit un bonus de vitesse, soit un bonus de revenu.

Dans tous les cas, on décrémentera le nombre d'anges actifs du coût de l'upgrade.

On n'oubliera pas également d'appeler le point d'accès serveur qui correspond à la mutation :

```
mutation AcheterAngelUpgrade($user: String! , $name: String!) {
  acheterAngelUpgrade(user:$user, name: $name) {
    name
  }
}
```

Finalisation et branchement sur un autre monde

Finaliser votre monde en donnant les spécifications complète de six produits et en testant que le jeu est intéressant du point de vue de la croissance des revenus (qui ne doit être ni trop rapide, ni trop lente).

Essayez également de vous brancher sur un autre monde, par exemple un monde conçu par un de vos camarades. Il vous faut pour cela obtenir l'adresse du serveur hébergeant ce monde, et de modifier l'adresse du serveur dans les fichiers `graphql.provider.ts` et `GraphService.ts` pour que votre client s'y connecte. Si les deux parties sont compatibles, les communications devraient fonctionner correctement.

Si de plus les codes sont exempts de bugs, le score calculé par ce serveur, et le score calculé par le client devrait être *relativement* identique, le *relativement* venant du fait qu'il est normal qu'une certaine dérive puisse apparaître avec le temps, du fait de micro-différences temporelles entre les calculs du client et du serveur. Au final c'est le serveur qui fait foi puisque c'est sur lui que se recalcule le client à chaque rechargement du jeu.

Vous êtes allés au bout du projet, félicitation !

Vous avez fait preuve de solides compétences en développement en ce qui concerne les langages Java, TypeScript, HTML et CSS. Vous avez créé des services web au format GraphQL et manipulant des représentations au format JSON. Vous avez manipulé la plateforme node.js et les *frameworks* web Angular et Material Design.

Vous avez de plus exercé vos compétences en calculs mathématiques et avez travaillé sur des problématiques de synchronisation temporelle entre un serveur et ses clients.